

# Zero-Case Degeneracy in Lynxer’s QuickSilver Relation Enables Universal Forgery

A Public-Key-Only Forgery Against Six Submitted Parameter Sets

Martin Feussner  
Selmer Center, University of Bergen  
martin.feussner@uib.no

27 September 2026

**Provenance and review status.** The attack was independently found by an AI system, OpenAI Codex using Daybreak Blue at maximum reasoning effort, without a human first proposing this specific attack. The derivation, implementation, experiments, and initial manuscript were also produced by the AI. At the date above, no human cryptanalyst has independently verified the argument, implementation, measurements, or novelty assessment. This is a preliminary disclosure for human review and reproduction.

## Abstract

Lynxer is a VOLE-in-the-head signature submitted to the Next-generation Commercial Cryptographic Algorithms Program. Its signing proof is intended to establish knowledge of a preimage of the custom one-way function Lynx. For security parameters  $\lambda \in \{256, 384, 512\}$ , however, the three quadratic QuickSilver constraints admit a publicly computable degenerate witness for every public key. Set the purported Lynx key to the public constant  $c_3$ , set the first S-box output to zero, and evaluate the second S-box according to the specified relation. The first constraint then vanishes because every term contains the first S-box output. The output constraint also vanishes because its two products contain, respectively, that zero output and the factor  $k + c_3 = 0$ . The public target is therefore left unconstrained.

This yields a zero-query public-key-only existential forgery for Lynxer-256s, Lynxer-256f, Lynxer-384s, Lynxer-384f, Lynxer-512s, and Lynxer-512f. The attack does not require a Lynx preimage, a break of any symmetric primitive, malformed input, or verifier modification. A reproducer linked against the submitted reference sources generated signatures accepted by all six unmodified verifiers. In every experiment, direct Lynx evaluation confirmed that the constructed key was not a preimage of the victim target. On an Intel Core i5-1135G7 laptop, witness construction took at most 1.333 seconds, while complete proof generation took between 0.459 and 12.029 seconds in the tested portable C builds. The two 160-bit parameter sets use a different output relation and are outside the scope of this attack.

## 1 Introduction

Lynxer turns a proof of knowledge for the symmetric primitive Lynx into a signature using the VOLE-in-the-head and Fiat–Shamir methodology. A public key contains an initial vector  $\text{iv}_{\text{owf}}$  and a target  $t$ . A signature is meant to prove knowledge of a key  $k$  such that

$$\text{Lynx}_{\text{iv}_{\text{owf}}}(k) = t.$$

The prover commits to an extended witness  $(k, v_1, v_2)$  and uses QuickSilver to prove three quadratic constraints. Consequently, security requires those constraints to describe the intended Lynx input–output relation.

For the six parameter sets with  $\lambda \geq 256$ , they do not. Two independently under-constrained zero cases align. First, the low-degree constraint for the first power map accepts  $v_1 = 0$  for every  $k$ , although the intended S-box output is generally nonzero. Second, the final inverse is checked only after multiplication by  $k + c_3$ . At  $k = c_3$  this check forgets the public output  $t$ . Choosing both values at once leaves only the second S-box constraint, which the attacker can satisfy by ordinary public evaluation.

The resulting attack is a direct forgery rather than a reduction in a security estimate. It requires no signing-oracle transcript, and its computational work is comparable to that of one honest signature. This note makes four contributions:

1. it gives an explicit false witness for every public target at  $\lambda \in \{256, 384, 512\}$ ;
2. it proves algebraically that all three submitted QuickSilver constraints vanish;
3. it turns the witness into a complete fresh-message forgery using the submitted prover and unmodified public verifier; and
4. it supplies a compact reproducer, source-archive hash, and laptop measurements for all six affected slow and fast parameter sets.

This attack concerns the relation specified for Lynxer. It is not a generic attack on QuickSilver, VOLE-in-the-head, or the intended one-wayness of Lynx. The 160-bit instances have different first and output constraints and are not claimed to be broken here.

## 2 The intended Lynx relation

Let  $\mathbb{F} = \mathbb{F}_{2^\lambda}$  and write addition in  $\mathbb{F}$  as  $+$ . It is therefore also XOR. The public initial vector expands to binary linear maps  $L_0, L_1, L_2, L_3$  and constants  $c_1, c_2, c_3 \in \mathbb{F}$  for  $\lambda \in \{256, 384, 512\}$ . The submitted C code uses zero-based names `C0`, `C1`, and `C2`; thus its `C2` is the specification's  $c_3$ .

Given a secret  $k \in \mathbb{F}$ , the witness-extension procedure computes

$$v_1 = S_{1,\lambda}(k + c_1), \tag{1}$$

$$x = L_0(k) + c_2, \tag{2}$$

$$v_2 = S_{2,\lambda}(x). \tag{3}$$

Both S-boxes are power maps. The relevant exponents are reproduced in Table 1. The first exponent is the inverse modulo  $2^\lambda - 1$  of the displayed verification exponent.

Table 1: Nonlinear maps in the affected parameter sets.

| $\lambda$ | relation defining $S_{1,\lambda}$ | $S_{2,\lambda}(x)$     |
|-----------|-----------------------------------|------------------------|
| 256       | $v_1^{2^5-1} = k + c_1$           | $x^{2^{53}-1}$         |
| 384       | $v_1^{2^{13}-1} = k + c_1$        | $x^{2^{60}-2^{30}+1}$  |
| 512       | $v_1^{2^{12}-2^6+1} = k + c_1$    | $x^{2^{180}-2^{90}+1}$ |

Define

$$A = L_1(v_1)L_2(v_2), \quad W = L_3(k + v_1 + v_2).$$

The intended output for all three affected security parameters is

$$t = A(k + c_3)^{2^\lambda-2} + W. \tag{4}$$

Exponentiation by  $2^\lambda - 2$  is inversion on  $\mathbb{F}^*$  and maps zero to zero under the polynomial convention used by the implementation. For the affected parameter sets, honest key generation rejects the case  $k = c_3$ ; however, that domain restriction is not enforced by the QuickSilver relation checked during verification.

### 3 The checked quadratic relation

The extended witness is  $(k, v_1, v_2) \in \mathbb{F}^3$ . The first QuickSilver polynomial is

$$z_{1,256} = v_1(k + c_1) + v_1^{2^5}, \quad (5)$$

$$z_{1,384} = v_1(k + c_1) + v_1^{2^{13}}, \quad (6)$$

$$z_{1,512} = v_1^{2^6}(k + c_1) + v_1^{2^{12}}v_1. \quad (7)$$

For nonzero  $v_1$ , division by  $v_1$  or  $v_1^{2^6}$  recovers the intended power-map relation. For  $v_1 = 0$ , every displayed polynomial is zero independently of  $k$ .

The second polynomial checks the  $S_2$  computation. In the specification, it is

$$z_{2,256} = x^{2^{53}} + xv_2, \quad (8)$$

$$z_{2,384} = x^{2^{60}}x + x^{2^{30}}v_2, \quad (9)$$

$$z_{2,512} = x^{2^{180}}x + x^{2^{90}}v_2. \quad (10)$$

In each row, substituting  $v_2 = S_{2,\lambda}(x)$  gives zero. The 256-bit reference source multiplies the first displayed equation by an additional  $x$ ; the same substitution still gives zero, so this implementation detail does not affect the attack.

Finally, rather than checking Equation (4) directly, Lynxer clears the inverse denominator and checks

$$z_3 = A + (W + t)(k + c_3) = 0. \quad (11)$$

When  $k + c_3 \neq 0$ , this equation is equivalent to Equation (4). When  $k = c_3$ , it reduces to  $A = 0$  and contains neither  $W$  nor the public target  $t$ .

Equations (5)–(11) are the relation proved by QuickSilver. They do not enforce either  $v_1 \neq 0$  or  $k + c_3 \neq 0$ . Thus the proved relation strictly contains the intended witness relation.

### 4 Public construction of a false witness

The attack uses only values derived from the public initial vector.

**Theorem 1** (Universal false witness). *Fix  $\lambda \in \{256, 384, 512\}$ , any public initial vector, and any target  $t \in \mathbb{F}$ . Let the resulting public values be  $L_0, L_1, L_2, L_3, c_1, c_2, c_3$ . Define*

$$k^* = c_3, \quad v_1^* = 0, \quad v_2^* = S_{2,\lambda}(L_0(c_3) + c_2). \quad (12)$$

*Then  $(k^*, v_1^*, v_2^*)$  satisfies all three Lynxer QuickSilver constraints for  $t$ .*

*Proof.* Every term of  $z_{1,\lambda}$  contains a positive power of  $v_1^*$ . Hence  $z_{1,\lambda} = 0$  for all three values of  $\lambda$ . The value  $v_2^*$  is the honest evaluation of the second S-box at  $x^* = L_0(k^*) + c_2$ , so substitution in the appropriate  $z_{2,\lambda}$  gives zero by construction.

It remains to check the output polynomial. Linearity gives  $L_1(v_1^*) = L_1(0) = 0$ , while  $k^* + c_3 = c_3 + c_3 = 0$  in characteristic two. Therefore

$$z_3 = L_1(0)L_2(v_2^*) + (L_3(c_3 + 0 + v_2^*) + t)(c_3 + c_3) = 0 + 0 = 0.$$

The calculation is independent of  $t$ . □

The triple is generally not an extended witness for the intended Lynx evaluation. Since  $S_{1,\lambda}$  is a power permutation with  $S_{1,\lambda}(0) = 0$ , its intended first intermediate value at  $k^* = c_3$  equals zero only if  $c_3 + c_1 = 0$ . Under the random-XOF model, this exceptional equality occurs with probability  $2^{-\lambda}$ . Direct evaluation in every experiment below also showed

$$\text{Lynx}_{\text{iv}_{\text{owf}}}(c_3) \neq t.$$

#### 4.1 From the witness to a signature forgery

Theorem 1 immediately yields a zero-query EUF-CMA adversary:

1. Parse the victim public key as  $(\text{iv}_{\text{owf}}, t)$  and expand the public Lynx matrices and constants.
2. Construct  $(k^*, v_1^*, v_2^*)$  with Equation (12).
3. On any fresh message  $M$ , run the public Lynxer/VOLE-in-the-head prover with that witness and fresh prover randomness.
4. Return the resulting ordinary signature.

By completeness, verification accepts because the committed witness satisfies exactly the three polynomials checked by the verifier. The adversary never invokes a signing oracle and never uses the victim secret key. Any probabilistic proof-generation or grinding step can be repeated exactly as in honest signing.

**Corollary 2.** *Under the completeness of the submitted proof implementation, Lynxer-256s, Lynxer-256f, Lynxer-384s, Lynxer-384f, Lynxer-512s, and Lynxer-512f are existentially forgeable from a public key alone.*

## 5 End-to-end validation

### 5.1 Provenance and method

The reproducer was tested against the official Round 1 Lynxer archive obtained from the NICCS candidate page. The archive used in the experiment had size 17,106,428 bytes and SHA-256 digest

34d863f7df8979c4a5e27fcfe657ed54e81905115c0afc8ef749f50af432f928.

No submitted source file was edited. The attack driver was linked against each reference implementation directory, excluding only `KAT_SIG.c` because it defines a competing `main` function.

For each parameter set, the driver performs the following checks.

1. It calls the official `sig_keygen` to create a victim key pair, then overwrites the secret-key buffer before constructing the attack witness.
2. It calls `lynx_init`, copies the third public constant, calls the official witness extension only to evaluate  $v_2$ , and replaces  $v_1$  by zero.
3. It directly evaluates the official Lynx OWF at  $k^*$  and checks that the result is different from the public target.
4. It calls the submitted `voleith_sign` proof routine with the forged witness and a message that has never been signed.
5. It submits the resulting byte string to the unmodified public `sig_verify`.

Calling the proof-generation routine directly does not provide an oracle and does not modify the verifier. It invokes the submitted implementation of the public prover logic after the attacker has constructed a witness for the checked relation. All commitment, VOLE consistency, Fiat–Shamir, grinding, parsing, and verification code remains unchanged from the submission.

## 5.2 Results

Table 2 reports one deterministic run per submitted parameter set. The host was a laptop with an 11th-generation Intel Core i5-1135G7 at 2.40 GHz, running 64-bit Windows. The portable reference sources were compiled with GCC 8.3.0, -O2, C99, and the submission’s XOF\_PSEUDO backend. Times are wall-clock milliseconds as reported by the driver. These timings demonstrate feasibility and are not intended as optimized benchmarks.

Table 2: Public-key-only forgeries against the unmodified submitted verifiers.

| Parameter   | $ pk $<br>bytes | $ \sigma $<br>bytes | witness<br>ms | proof<br>ms | verify<br>ms | accepted |
|-------------|-----------------|---------------------|---------------|-------------|--------------|----------|
| Lynxer-256s | 64              | 12,191              | 87            | 3,856       | 3,680        | yes      |
| Lynxer-256f | 64              | 15,097              | 109           | 459         | 463          | yes      |
| Lynxer-384s | 96              | 27,495              | 450           | 12,029      | 11,656       | yes      |
| Lynxer-384f | 96              | 34,109              | 502           | 1,790       | 1,405        | yes      |
| Lynxer-512s | 128             | 48,879              | 1,333         | 1,431       | 1,646        | yes      |
| Lynxer-512f | 128             | 61,091              | 1,317         | 715         | 661          | yes      |

For all six runs, the verifier returned its success value zero, while direct OWF evaluation confirmed that  $k^*$  was not a preimage of the victim target. The largest measured time to construct the witness and produce a complete signature was 12.479 seconds. Memory use is essentially that of one ordinary reference signer plus the small attack driver; the attack stores no signature transcript and performs no search.

A separate clean rerun used GCC 13.3.0 with -O2 -march=native on a four-core Intel Core i5-6500 running 64-bit Linux. After victim key generation, the rerun erased the secret-key buffer and reset the random generator to a separate public attacker seed before constructing the witness. All six forged signatures again verified, direct Lynx evaluation again showed that  $k^*$  was not the victim preimage, and every signature was rejected after changing the message. Proof-generation CPU times ranged from 0.467 to 11.371 seconds and verification CPU times from 0.461 to 11.081 seconds. This second pass was also performed within the AI audit and is not independent human verification.

## 5.3 Reproduction

The attack description is self-contained. A direct reproduction against the submitted reference sources proceeds as follows. Extract the archive whose size and digest appear above. For each affected reference directory, link all submitted C sources except KAT\_SIG.c with a small driver that:

1. calls `sig_keygen` and retains only the resulting public key;
2. expands the public matrices and constants with `lynx_init`;
3. copies `C2` into the first witness field, zeros the second field, and obtains the third field by evaluating the submitted second S-box;
4. invokes `voleith_sign` on a fresh message with attacker-chosen randomness; and
5. passes the resulting signature to the unchanged `sig_verify`.

For a negative control, evaluate the ordinary Lynx function at **C2** and compare it with the public target; under the random-XOF model they coincide only with negligible probability. Changing the signed message should also make verification fail. These steps use no value from the victim secret key after public-key generation.

## 6 Security impact and scope

The public target  $t$  is intended to bind a signature to the victim public key. Equation (11) loses that binding for the witness in Equation (12). Consequently, the attack has the following properties.

- **No secret recovery is needed.** The alleged key is a public constant.
- **No chosen-message transcript is needed.** The attack makes zero signing queries and directly signs a fresh message.
- **The target is arbitrary.** The same construction works for every  $t$  because  $t$  is multiplied by zero in the checked relation.
- **The encoding is ordinary.** The final byte string is generated and parsed by the submitted code with the documented length.
- **The attack is mathematical.** Equations (5)–(11) occur in the specification itself. The attack is not based on undefined behaviour, entropy failure, memory corruption, or a parser discrepancy.

The attack covers exactly the six submitted parameter sets whose Lynx output contains the additional  $(k + c_3)^{-1}$  layer. Lynxer-160s and Lynxer-160f instead check  $v_1(k + c_1) = 1$  and use  $t = L_1(v_1)L_2(v_2) + L_3(k + v_1 + v_2)$  without that denominator. The witness above therefore does not satisfy their relation. No security claim about the 160-bit sets follows from this paper.

The attack does not establish that the intended Lynx function is easy to invert. That distinction does not preserve the affected signature instances: Lynxer proves the relaxed polynomial relation, whereas signature security requires knowledge soundness for the intended preimage relation.

## 7 Cause and repair

The failure arises from the composition of two under-constrained polynomializations.

First, the intended first S-box relation is obtained from the checked quadratic equation only after division by  $v_1$  (or by  $v_1^{2^6}$  at  $\lambda = 512$ ). The quadratic equation has the extra component  $v_1 = 0$ . Second, Equation (11) is obtained from the inverse output formula only after multiplication by  $k + c_3$ . It has an extra component at  $k = c_3$ . Their intersection contains the efficiently constructible witness of Theorem 1 for every public output.

A repair must make the proved relation exactly match the witness-extension and output evaluation relations, including all zero cases. Merely rejecting  $k = c_3$  in honest key generation is insufficient: the malicious prover chooses the witness. Straightforward options include adding auxiliary inverse witnesses and constraints such as

$$r_1 v_1 = 1, \quad r_3(k + c_3) = 1,$$

together with corresponding key-generation restrictions, or redesigning the S-box and output constraints to encode their complete graphs without division. The second S-box constraints should receive the same zero-case review. Any repair changes the relation and potentially the

witness length, proof cost, test vectors, and security proof; it therefore requires a new full analysis rather than a local verifier-side input check.

As a general design rule, clearing a denominator in a proof circuit preserves equivalence only when nonzeroness of that denominator is itself proved. Power-map identities obtained by factoring out the alleged output require the same care.

## 8 Novelty review and limitations

This work was derived independently from the submitted specification and source. A public search performed on 27 September 2026 found the ngcc.dev report index listing Lynxer (candidate `sign-14`) as having no report [5]. Searches for Lynxer cryptanalysis and for the specific QuickSilver degeneracy found the design paper [2], but no earlier disclosure of this forgery. This is a time-bounded novelty check, not evidence that no private report, forum comment, or unindexed analysis exists.

The experiment uses one deterministic victim per parameter set to validate the algebra and the submitted verification paths. Because Theorem 1 is algebraic and applies to every public target, these runs are not intended to estimate a key-dependent attack success probability. Only the assertion that the constructed witness lies outside the intended witness-extension relation has the negligible public-constant exception discussed above. The timing table is specific to one portable Windows build and should not be read as a performance comparison with the authors’ optimized implementation.

## 9 Conclusion

For each submitted Lynxer parameter set at 256, 384, and 512 bits, the alleged witness

$$(c_3, 0, S_{2,\lambda}(L_0(c_3) + c_2))$$

satisfies all three QuickSilver constraints for every public output. This converts public information into an accepted fresh-message signature in seconds on a laptop. The break arises from missing zero-case constraints at two algebraic divisions and invalidates the intended proof-of-preimage statement independently of the assumed strength of the Lynx primitive.

## References

- [1] K. Yang, Y. Yu, L. Jiao, H. Cui, X. Guo, Q. Zheng, J. Zhang, H. Yang, Y. He, Y. Hao, C. Guo, and J. Ge. *Lynxer: A VOLE-in-the-Head Signature Scheme based on Only Symmetric-key Primitives: Algorithm Specifications and Supporting Documentation*, Version 1.1, NGCC Round 1 submission, 2026. [Official NICCS Lynxer archive](#).
- [2] L. Jiao, H. Yang, H. Cui, Y. He, Y. Hao, X. Guo, Q. Zheng, J. Zhang, Y. Yu, and K. Yang. Lynx: Symmetric Primitive for Shorter and Faster VOLE-in-the-Head Signatures. *Cryptology ePrint Archive*, Paper 2026/1099, 2026. <https://eprint.iacr.org/2026/1099>.
- [3] K. Yang, P. Sarkar, C. Weng, and X. Wang. QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field. In *ACM CCS 2021*, pages 2986–3001, 2021.
- [4] C. Baum, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, and P. Scholl. Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head. In *CRYPTO 2023, Part V*, pages 581–615, 2023.

- [5] ngcc.dev. NGCC reports index, snapshot consulted 27 September 2026. <https://ngcc.dev/reports/>.